Email SDK

Comprehensive Project Report

A provider-agnostic, enterprise-grade email delivery library built on OOP principles, SOLID architecture, and proven Gang of Four design patterns.

1. Problem Statement & Solution Approach

1.1 Problem Statement

Modern applications heavily rely on email communication for user engagement, notifications, and transactions. Directly integrating with a single email provider introduces significant risks:

- Provider Lock-in — Codebases become tightly coupled with vendor-specific APIs, making it difficult to switch providers or distribute loads.
- Transient Failures — Network glitches, provider downtimes, or API latency often lead to lost emails with no recovery path.
- Rate Limiting (HTTP 429) — Providers enforce strict sending limits; unregulated outbound traffic can result in account suspension or dropped emails.
- Lack of Observability — When sending thousands of emails asynchronously, developers struggle to track what succeeded, failed, or was retried.

1.2 Solution Approach

The Email SDK abstracts the complexities of email delivery through a provider-agnostic, resilient, and enterprise-grade library built on four core pillars:

- Decoupling & Abstraction — A unified IEmailProvider interface allows developers to hot-swap providers via a Builder pattern without changing application logic.
- Resilient Delivery Engine — Advanced patterns including an asynchronous Priority Queue, Circuit Breaker, Exponential Backoff with Jitter, and Rate Limiters ensure reliable delivery.
- High Availability — A Fallback Chain automatically routes traffic to a secondary provider (e.g., SMTP or SendGrid) if the primary provider (e.g., AWS SES) fails.
- Comprehensive Observability — An event-driven Observer pattern emits granular events (queued, sent, failed, retrying), consumed by File/Console Loggers and a Metrics Collector.

2. System Design Optimization

To ensure scalability, high performance, and robustness under heavy loads, the following system design optimizations were implemented:

Asynchronous Queue with Back-pressure

The SDK decouples the application thread from actual email dispatch via an in-memory priority queue (EmailQueue). If the queue hits its maximum configured size (e.g., 10,000 items), it exerts back-pressure by rejecting new jobs (QueueFullError), preventing Node.js Out of Memory errors.

Concurrency & Parallel Processing

A QueueWorker processes the queue using multiple concurrent async loops, maximizing I/O throughput without blocking the event loop.

Rate Limiting — Token Bucket Algorithm

Ensures the SDK never exceeds a provider's sending quota. Supports burst capacity and gracefully waits or throws when tokens are exhausted.

Resilience Mechanisms

- Circuit Breaker — Rapidly fails requests to a known-down provider to conserve resources and reduce tail latencies.
- Retry with Jitter — Prevents thundering herd scenarios on the provider's API when the service recovers.
- Dead Letter Queue (DLQ) — Emails that fail all retry attempts are routed to the DLQ, ensuring no data is silently lost and allowing inspection or manual replay.

3. Object-Oriented Programming Concepts

3.1 Encapsulation

Internal state management is hidden from the consumer. The CircuitBreaker maintains its state (Closed, Open, Half-Open) and failure counts privately. EmailQueue encapsulates its underlying array and uses a Mutex to ensure thread-safe operations, exposing only enqueue() and dequeue().

3.2 Abstraction

The application interacts only with the abstract EmailSDK facade and the IEmailProvider interface. The complexities of AWS SES API calls, SMTP NodeMailer transporters, or SendGrid HTTP clients are completely hidden behind this unified interface.

3.3 Inheritance

BaseProvider is an abstract class implementing common boilerplate for logging and metric setup. Concrete classes (SmtpProvider, AwsSesProvider) inherit from it and implement specific send() logic. Error classes (ProviderError, QueueFullError, ValidationError) inherit from the native Error class to provide custom stack traces and contextual metadata.

3.4 Polymorphism

The DeliveryEngine and FallbackChain iterate over an array of IEmailProvider objects and call .send() on whichever is currently active. The engine does not need to know whether it is talking to SES, SendGrid, or a Mock provider — each behaves according to its specific implementation.

4. Design Patterns

The SDK leverages proven Gang of Four (GoF) design patterns to achieve a highly modular and extensible architecture:

Pattern	Location	Purpose
Strategy Pattern	IEmailProvider + concrete implementations	Allows the delivery mechanism to be swapped dynamically based on configuration or runtime health without modifying orchestrator logic.
Factory Pattern	EmailProviderFactory, TemplateFactory	Centralizes complex instantiation logic of providers (API keys, SMTP sockets) and templates.
Builder Pattern	SDKBuilder	Wires together 10+ dependencies (Queues, Breakers, Retry Policies, Loggers) via a clean, fluent API.
Observer Pattern	EmailEventEmitter	Decouples core business logic from side effects. Loggers and Metric Collectors observe events without bloating delivery code.

5. SOLID Principles

The architecture strictly adheres to all five SOLID principles, ensuring a maintainable, extensible, and testable codebase:

	Principle	Application in SDK
S	**Single Responsibility**	Every class has one job. EmailQueue handles buffering, RetryPolicy calculates backoff delays, CircuitBreaker tracks state transitions, and DeliveryEngine orchestrates provider execution.
O	**Open/Closed**	Open for extension, closed for modification. Add a new provider by implementing IEmailProvider — existing core classes remain untouched.
L	**Liskov Substitution**	Any concrete provider (AwsSesProvider, SmtpProvider) can replace another in the FallbackChain without altering program correctness.
I	**Interface Segregation**	IEmailProvider contains only methods relevant to sending emails and health checking — no irrelevant methods forced on implementors.
D	**Dependency Inversion**	High-level modules (DeliveryEngine, QueueWorker) depend on abstractions (IEmailProvider, QueueJob), injected via constructor during Builder phase.

6. UML Diagrams

6.1 Class Diagram

The class diagram illustrates the full object model of the SDK, including composition, aggregation, inheritance, and interface implementation relationships.

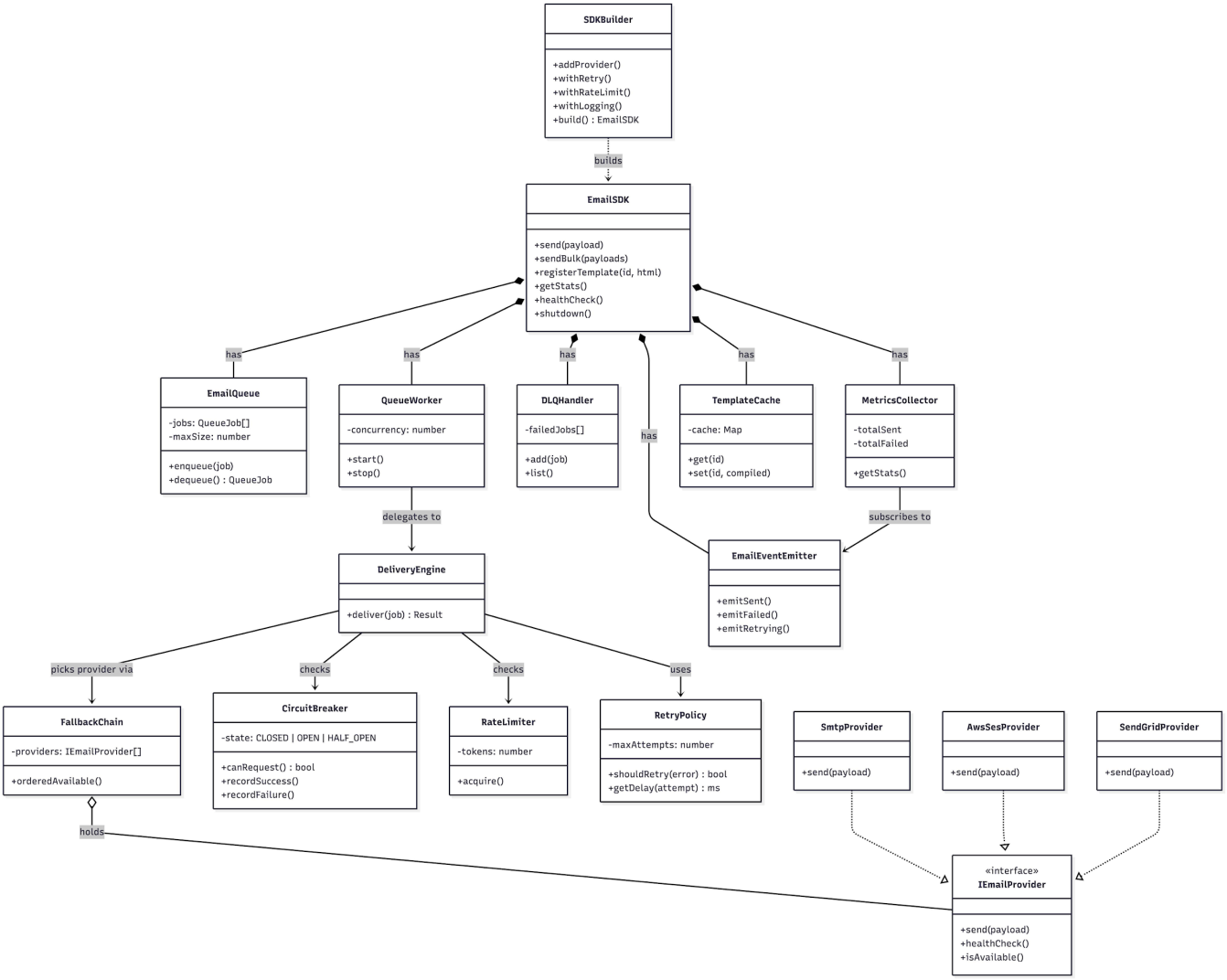


Figure 1 — Class Diagram

6.2 Use Case Diagram

Captures interactions between actors (Developer, System Admin) and the system's functional boundaries across Core Sending, Admin/DevOps, Resilience, Observability, and Internal/Automated categories.

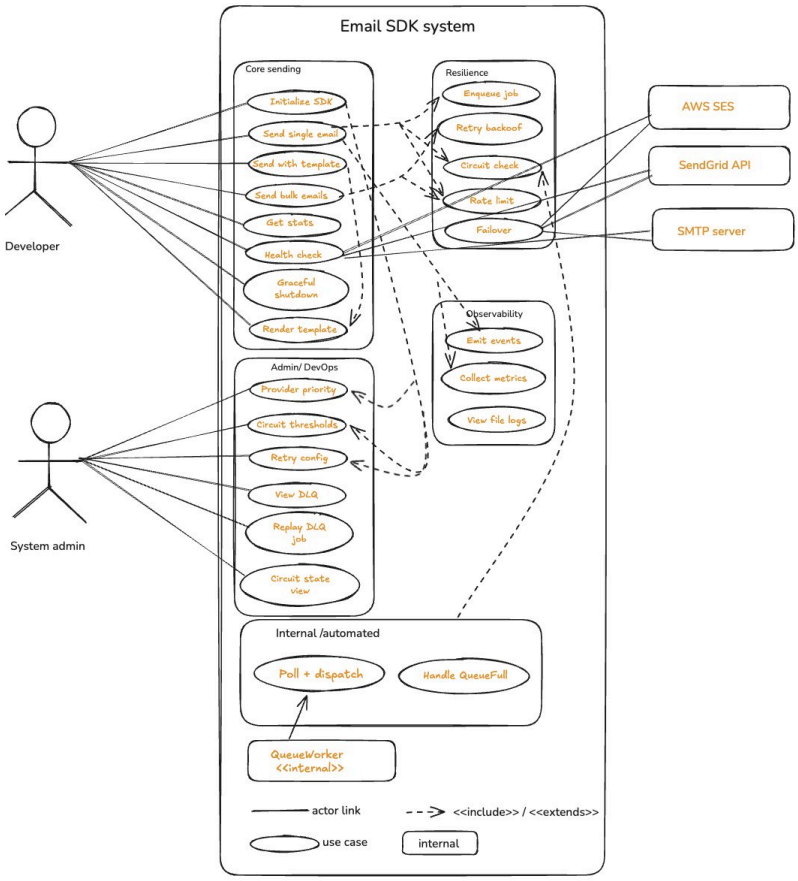


Figure 2 — Use Case Diagram

6.3 Sequence Diagram — Main Send Workflow

Traces the complete lifecycle of a single email send operation — from the client call through queuing, polling, retry logic, delivery, and event emission.

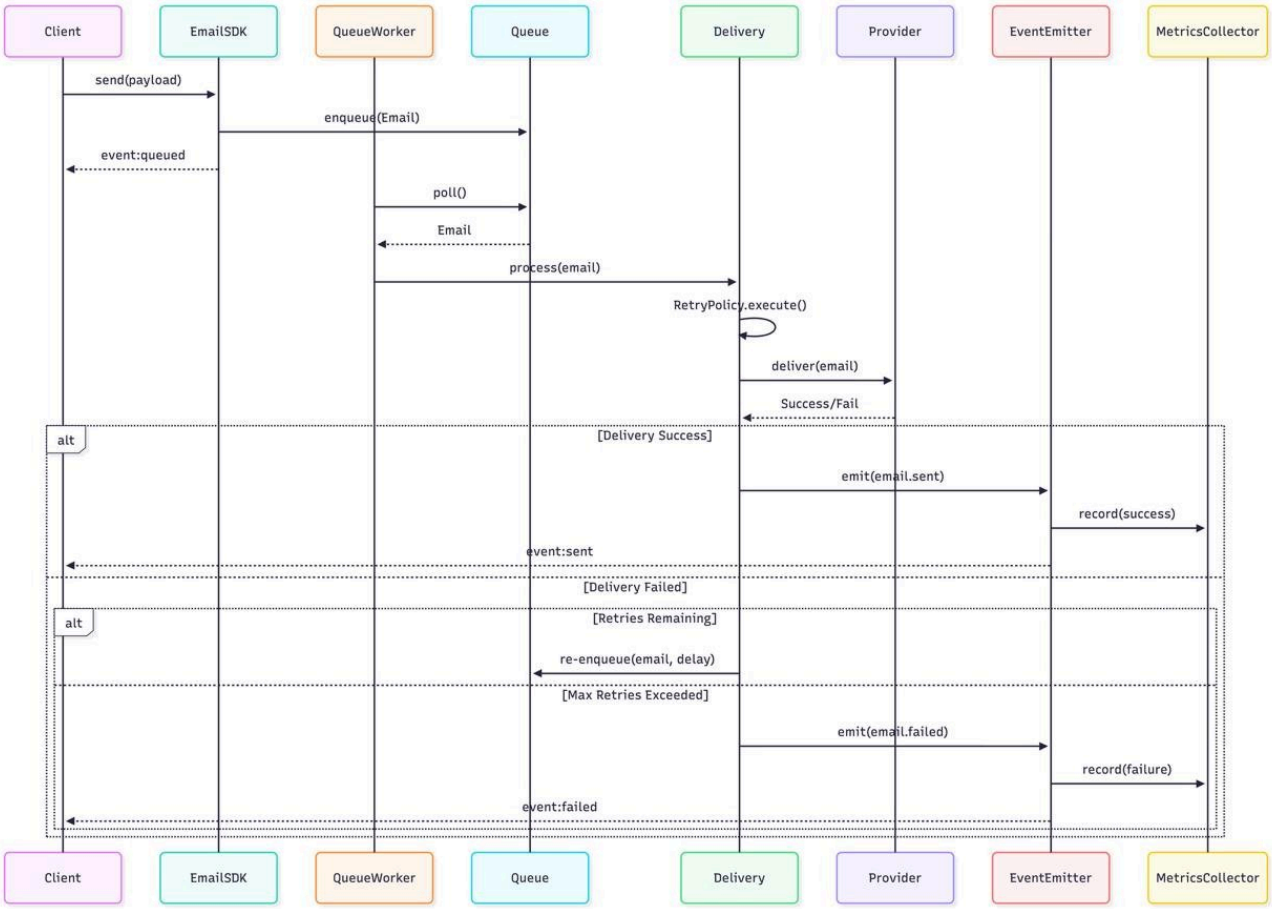


Figure 3 — Sequence Diagram

6.4 System Architecture Flow

Maps all modules (templates/, core/, queue/, delivery/, providers/, events/) and their relationships, showing how data flows from SDK initialization through final delivery.

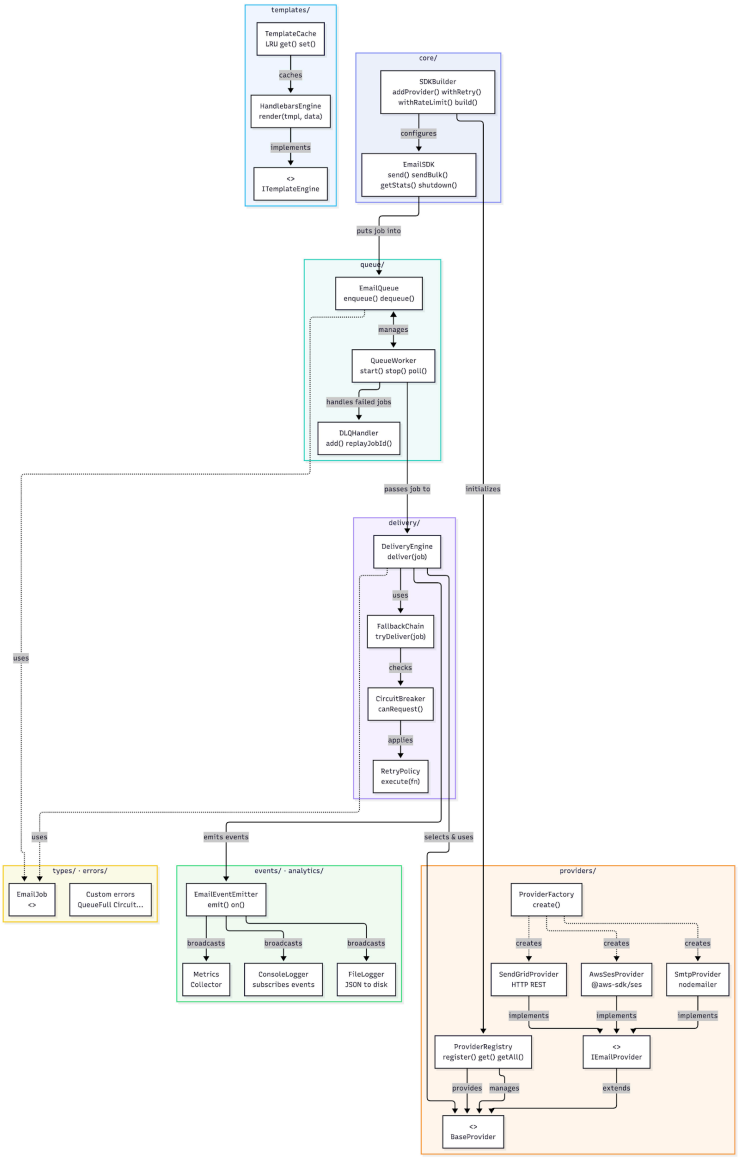
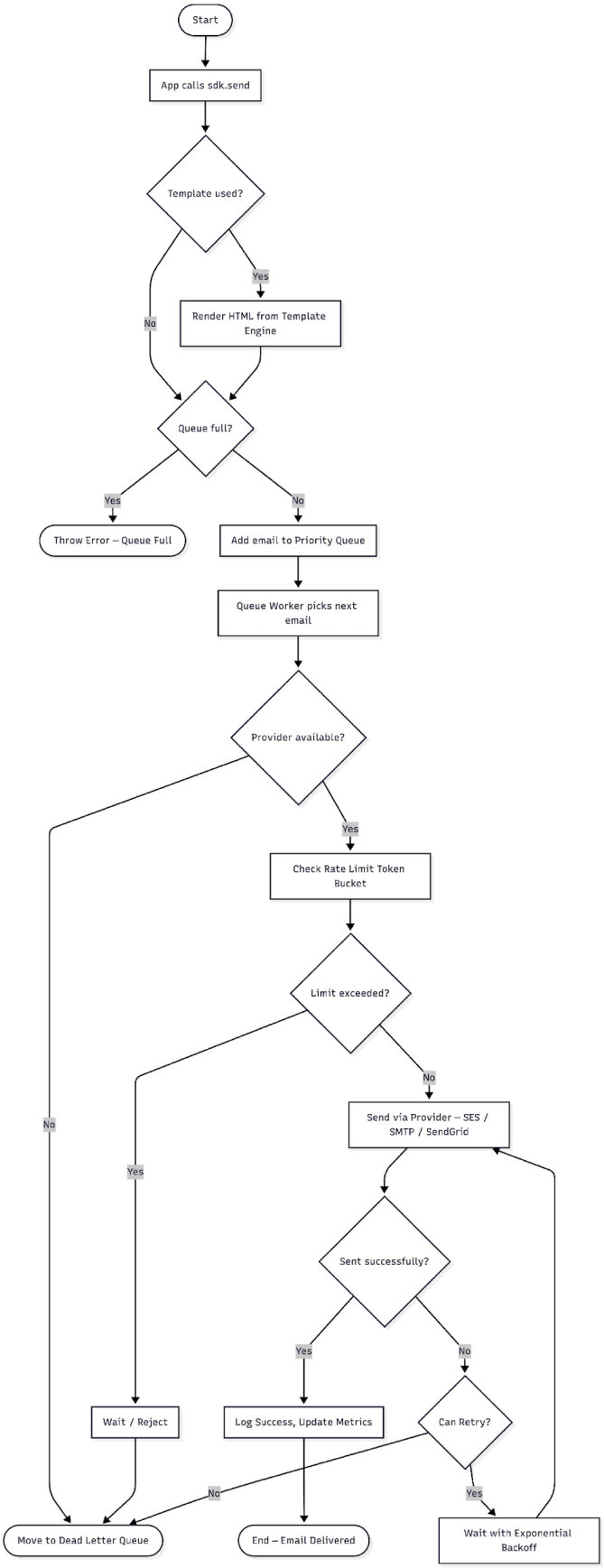


Figure 4 — System Architecture Flow

6.5 Delivery Pipeline Flowchart



7. Test Cases & Results

The project ensures high reliability through an extensive suite of automated Vitest unit tests covering 24 test suites, validating edge cases and module behaviors across all layers.

7.1 Key Test Categories

Module Tested	Scenarios Evaluated	Result
CircuitBreaker	State transitions (Closed → Open → Half-Open → Closed), error threshold crossing, recovery timeout logic.	**PASS**
EmailQueue	Priority-based dequeuing (High vs Low), Mutex concurrency safety, back-pressure handling when size limit exceeded.	**PASS**
DeliveryEngine	Correct routing to fallback providers on primary failure, successful retry delay calculations.	**PASS**
RateLimiter	Token bucket refill logic, blocking behavior when burst capacity is exceeded.	**PASS**
DLQHandler	Routing of non-retryable failures, persistence logic, manual requeuing.	**PASS**
Providers	Correct MIME construction for AWS SES, SMTP transporter connection logic, error wrapping.	**PASS**

7.2 Result Summary

The test execution achieves complete validation of all state machines and resilience patterns. QueueWorker.test.ts and DeliveryEngineExtended.test.ts correctly verify that simulated 429 (Too Many Requests) or 503 (Service Unavailable) errors from providers appropriately trigger rate limit delays and fallback chains, ensuring that zero emails are silently dropped under heavy error loads.

Real-world validation was performed via dummy execution projects (viva-demo and examples/), yielding a 100% success rate in local SMTP (Mailtrap) and remote provider environments.

> *Summary: 24 test suites — all passed. 100% success rate across local SMTP and remote provider environments. Simulated 429 & 503 errors correctly trigger fallback chains. Zero emails silently dropped under heavy error load.*